Computer Science & IT



C Programming

Array & Pointers

Lecture No. 01



By- Abhishek Sir

Recap of Previous Lecture



Topic — Activation tree

Topic — Storage class

Topic

Topic

Topic

Slide

Topics to be Covered



- Topic — Pointers
- Topic — Call by reference
- Topic
- Topic
- Topic

Slide

Pointers

With any variable 2-values are associate

L-value, r-value

int a = 10;



a - r-value = 10

a - l-value = 1000

Slide



Pointers

```
Scanf ("%d", &a);
```

& - address of (unary)

```
int a = 10;
printf ("%u", &a);
```

print address associated with variable a

& operator return address associated with variable.

Slide



Pointers

pointer is a variable which can hold address of another variable of Similar Data type

```
int a = 10;
```

```
int *p;
```
← declaration

* dereference operator



Pointers

int *p; p is pointer of integer type
p can hold address of an integer.

```
int a = 10;
int *p;
// pointer assigment
p = &a;
```

```
//pointer Inilization
int a = 10;
int *p = &a;
```

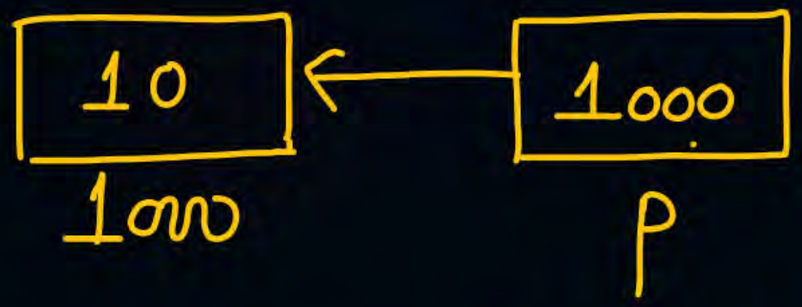
Slide



Pointers

p = &a;

Memory Diagram



a = a + 10;

10+10

we can Access variable a using pointer p

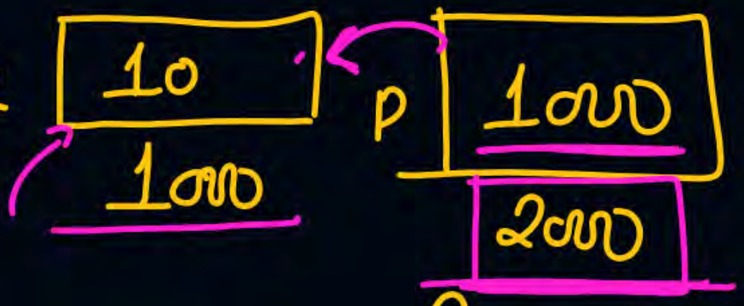
printf("%u", p); // Name of variable;

it will print addres store in p Locahon

Slide

Pointers





p value is 1000

```
int a=10;
int p=&a;
```

To access the location we need to perform dereference.

```
① printf("%d", *p);
② printf("%u", p);
③ printf("%u", &a);
④ printf("%d", a);
⑤ printf("%u", &p);
```

10	10	1000	10	2000
1000				
1000				1000
10				
2000				

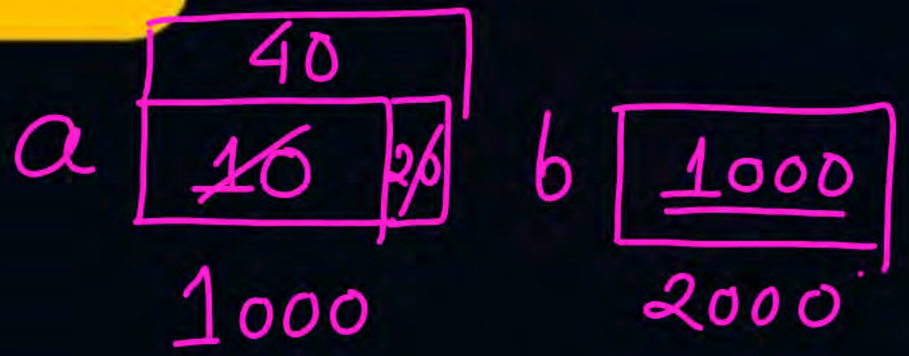
Slide



Pointers

```
#include<stdio.h>
int main (){
        int a, *b;
        a = 10;
        b = &a;
        a =a+10;
        *b = *b+20;
        printf("%d", a);
        return 0;
}
```

40



$*b = *b + 20$

$a = 20 + 20$

you can access variable through a as well as $*b$

Slide

```
int a=10;
int *p;
*p=20;
```



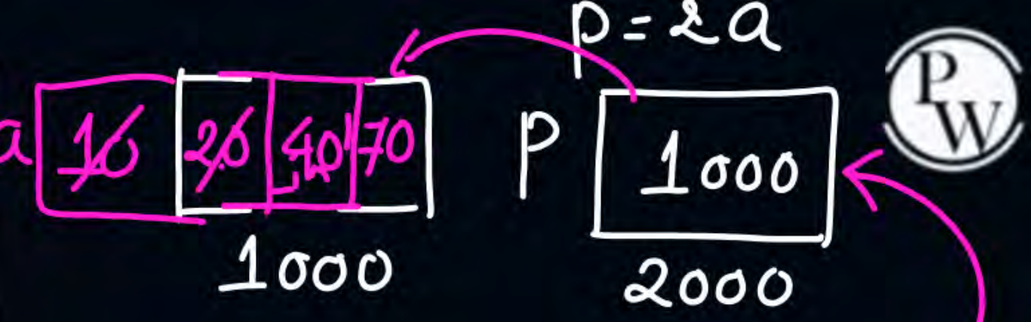
Uninitialized pointer, p contain Some garbage value, and in that value 20 is stored.



Pointers to Pointer

```
int a=10
int *p;
p=&a
a=a+10
*p=*p+20
```

a will update



q 2000

```
int a=10
int *p=&a;
int **q;
q=&p;
a=a+10;
*p=*p+20;
**q=**q+30
```

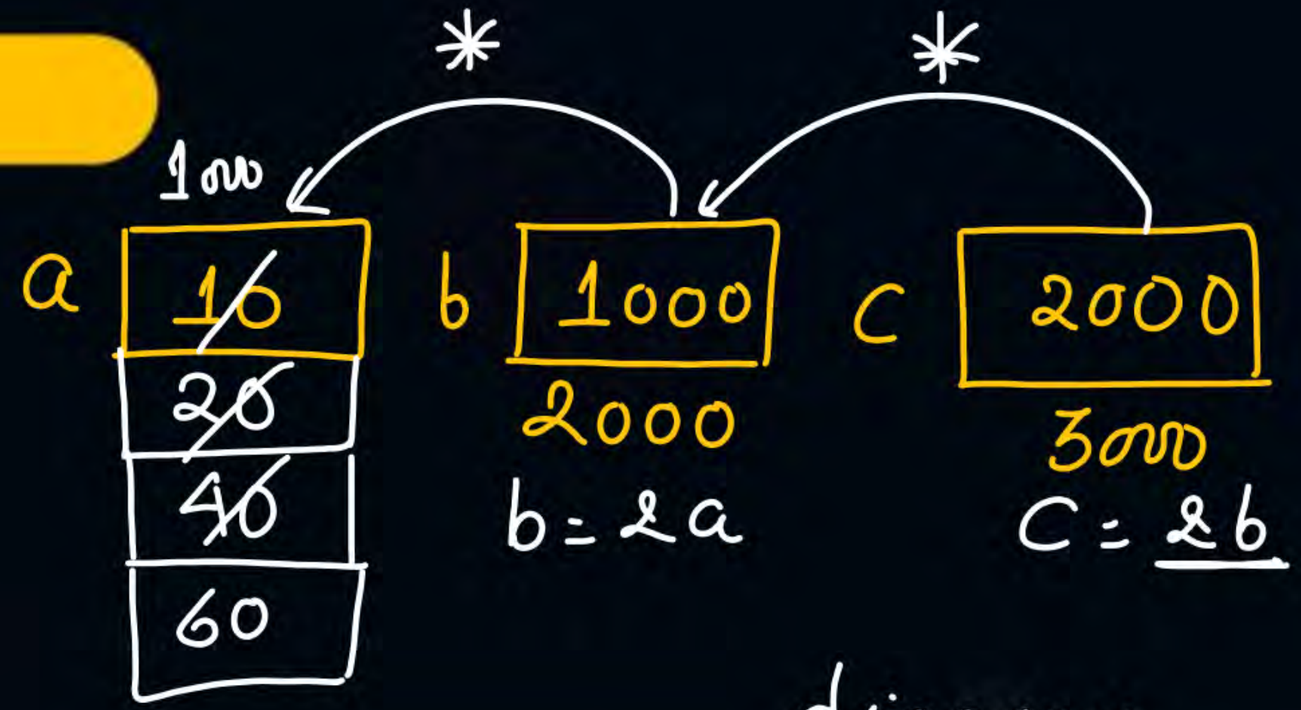
q=&p;

Slide

Pointers

```
#include<stdio.h>
int main (){
        int a, *b, **c;
        a = 10;
        b = &a;
        c = &b;
        a =a+10;
        *b = *b+20;
        **c = **c+20;
        printf("%d", a);
        return 0;
}
```

60



diagram

Slide



Size of Pointer

```
include <stdio.h>

int main(){
    int *p;
    char *p1;
    float *p2;

    printf("%lu", sizeof(p));
    printf("%lu", sizeof(p1));
    printf("%lu", sizeof(p2));

    return 0;
}
```



GATE 2017

printxy(1,1)

#Q Consider the following function implemented in C:

```
void printxy (int x, int y) {
        int *ptr ;
        x = 0;
        ptr = &x;
        y = * ptr;
        * ptr = 1;
        printf ("%d, %d," x, y);
    }
```

The output of invoking printxy (1, 1) is

(A) 0,0
(B) 0,1
(C) 1,0
(D) 1,1

```
# include<stdio.h>
int main() {
    printxy(1,1)
```

Slide



GATE 2017

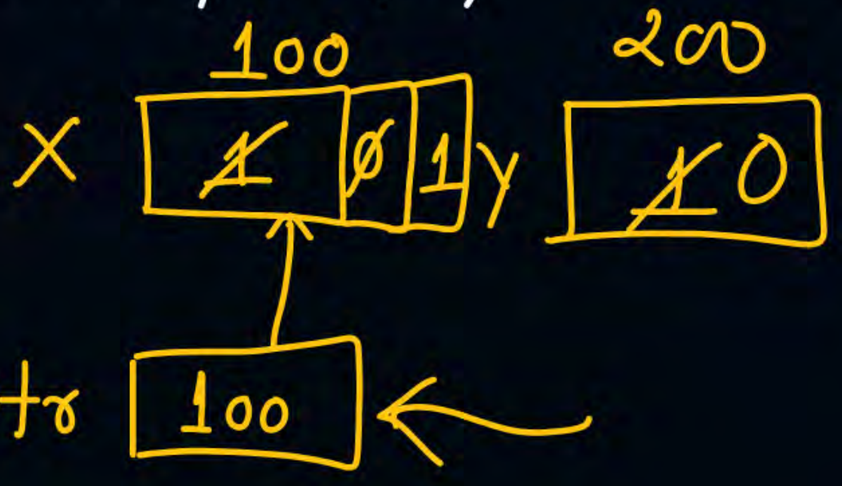
#Q Consider the following function implemented in C:

```
void printxy (int x, int y) {
        int *ptr ;
        x = 0;
        ptr = &x;
        y = * ptr;
        * ptr = 1;
        printf ("%d, %d," x, y);
}
```

y

1, 0

printxy(1,1)



y = *ptr

y = 0

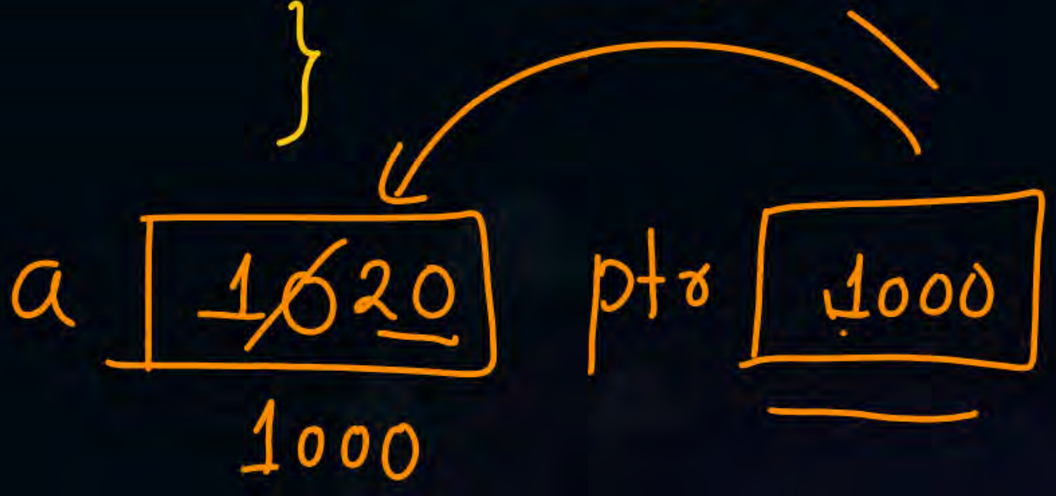
Slide

Call by reference

```
#include<stdio.h>
void fav(int*);
int main(){
    int a=10;
    fav(&a);
    printf("%d",a);
    return 0;
}
```

Output: 20

```
void fav(int *ptr){
    *ptr=*ptr+10;
}
```



Slide



Call by reference

a | 10 |

1000

*2a = a

*(1000)

= 10

Slide



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

pointer, &, *,

porter to pointer

Call by reference (started).

Slide

THANK - YOU

